

Code Explanation

Unit Tests for Batch Processing Module

The provided code is a unit test suite for a batch processing module, written using Pytest and PySpark. The tests validate the functionality of four main functions: `process\_customer\_data`, `process\_order\_data`, `create\_order\_summary\_scd2`, and `create\_customer\_aggregate\_spend`. However, the last two functions are not tested in the given code snippet.

The test suite creates sample customer and order data, processes it using the functions under test, and asserts that the results match the expected outcomes.

Overview of the Code

The code is designed to test the data processing logic for customer and order data. It uses PySpark to create DataFrames from sample data and then applies the processing functions to these DataFrames. The results are then validated against expected outcomes.

Step 1: Creating a Spark Session for Testing

The first step is to create a Spark session for testing purposes. This is achieved through a Pytest fixture named `spark`.

```
@pytest.fixture(scope="module")
def spark():
    """Create a Spark session for testing."""
    return SparkSession.builder
        .appName("Unit Tests")
        .master("local[1]")
        .getOrCreate()
```

Step 2: Creating Sample Customer and Order Data

The next step involves creating sample customer and order data. This is done using two separate Pytest fixtures: `sample\_customer\_data` and `sample\_order\_data`. Both fixtures create DataFrames with sample data and schemas.

```
@pytest.fixture(scope="module")
def sample_customer_data(spark):
    """Create sample customer data for testing."""
    data = [
        ("C001", "John Doe", "john@example.com", "North"),
        ("C002", "Jane Smith", "jane@example.com", "South"),
        ("C003", "Bob Johnson", "bob@example.com", "East"),
        (None, "Invalid", "invalid@example.com", "West"),
        ("C004", None, "missing@example.com", "North"),
        ("C001", "John Doe", "john@example.com", "North")
    ]

    schema = StructType([
        StructField("CustId", StringType(), True),
        StructField("Name", StringType(), True),
        StructField("EmailId", StringType(), True),
        StructField("Region", StringType(), True)
    ])
    return spark.createDataFrame(data, schema)
```

```
return spark.createDataFrame(data, schema)
```

```
@pytest.fixture(scope="module")
def sample_order_data(spark):
    """Create sample order data for testing."""
    data = [
        ("0001", "Laptop", 1000.0, 2, date(2023, 1, 15), "C001"),
        ("0002", "Phone", 500.0, 1, date(2023, 1, 20), "C002"),
        ("0003", "Tablet", 300.0, 3, date(2023, 1, 25), "C003"),
        ("0004", "Monitor", 200.0, 2, date(2023, 1, 30), "C001"),
        (None, "Invalid", 100.0, 1, date(2023, 2, 5), "C002"),
        ("0005", "Keyboard", 50.0, None, date(2023, 2, 10), "C003"),

        ("0001", "Laptop", 1000.0, 2, date(2023, 1, 15), "C001")
    ]

    schema = StructType([
        StructField("OrderId", StringType(), True),
        StructField("ItemName", StringType(), True),
        StructField("PricePerUnit", DoubleType(), True),
        StructField("Qty", IntegerType(), True),
        StructField("Date", DateType(), True),
        StructField("CustId", StringType(), True)
    ])

    return spark.createDataFrame(data, schema)
```

Step 3: Testing Customer Data Processing Logic

The `test\_process\_customer\_data` function tests the `process\_customer\_data` function. It mocks the `read` and `write` functions to return the sample customer data and to do nothing, respectively.

```
def test_process_customer_data(spark, sample_customer_data, monkeypatch):
    """Test customer data processing logic."""
    def mock_read(*args, **kwargs):
        return sample_customer_data

    class MockWriter:
        def format(self, *args, **kwargs):
            return self
        def mode(self, *args, **kwargs):
            return self
        def saveAsTable(self, *args, **kwargs):
            pass

    monkeypatch.setattr(spark, "read", mock_read)
    monkeypatch.setattr(sample_customer_data, "write", MockWriter().format)

    result_df = process_customer_data(spark)

    assert result_df.count() == 3
    assert len(result_df.columns) == 4
```

```

null_count = result_df.filter(
    F.col("CustId").isNull() |
    F.col("Name").isNull() |
    F.col("EmailId").isNull() |
    F.col("Region").isNull()
).count()
assert null_count == 0

distinct_count = result_df.select("CustId").distinct().count()
assert distinct_count == 3

```

Step 4: Testing Order Data Processing Logic

The `test\_process\_order\_data` function tests the `process\_order\_data` function in a similar manner to the customer data processing test.

```

def test_process_order_data(spark, sample_order_data, monkeypatch):
    """Test order data processing logic."""
    def mock_read(*args, **kwargs):
        return sample_order_data

    class MockWriter:
        def format(self, *args, **kwargs):
            return self
        def mode(self, *args, **kwargs):
            return self
        def saveAsTable(self, *args, **kwargs):
            pass

    monkeypatch.setattr(spark, "read", mock_read)
    monkeypatch.setattr(sample_order_data, "write", MockWriter().format)

    result_df = process_order_data(spark)

    assert result_df.count() == 4
    assert len(result_df.columns) == 7

    null_count = result_df.filter(
        F.col("OrderId").isNull() |
        F.col("ItemName").isNull() |
        F.col("PricePerUnit").isNull() |
        F.col("Qty").isNull() |
        F.col("Date").isNull() |
        F.col("CustId").isNull()
    ).count()
    assert null_count == 0

    distinct_count = result_df.select("OrderId").distinct().count()
    assert distinct_count == 4

```